

GradePath

A GPA and degree-class planner that tells a student what average they still need to reach the class they want. Laboratory Exercise 1 -Part 1, AI Web Application Challenge.

Group

Group name	#TeamFreeWill2.0
Project title	GradePath -GPA & Degree Class Planner
Date	4 September 2026

Members

NAME	INDEX NUMBER
Yasith Dharmasena	ICT/24/828
Anjana Ilangasinghe	ICT/24/858
Induwara Ranathunga	ICT/24/859
Sudeep Hirushan	ICT/24/862
Shanuka Lakshan	ICT/24/904

Problem, users and summary

The problem

Undergraduates track grades in scattered notes and spreadsheets and only discover late that a degree class has slipped out of reach. The arithmetic that matters is not the GPA already earned but the average still required over the credits left, and almost nobody works that out by hand. Repeat attempts, medical grades and non-GPA courses make the calculation easy to get wrong.

Intended users

Undergraduate students, first year through final year. The programme constants are set for the BICT degree at the University of Sri Jayewardenepura, and a single object at the top of the code changes them for any other 4.00-scale programme. Academic advisors can use the same page during consultations.

Summary

GradePath is a single-page web application that holds a student's transcript in their own browser. It shows the cumulative GPA, the degree class that GPA currently earns, the gap to the next class, and a trend of the cumulative GPA after each semester. The planner then answers the question the student

actually has: given a target class and the credits still to take, what average is needed, which grade is that, and is the target still reachable at all. No account, no server, no data leaving the device.

The essential user journey

1. Add a course: semester, code, credits and grade.
2. The cumulative GPA, degree class and credit counts update immediately.
3. Pick a target class and enter the credits still to be taken.
4. Read the verdict: the average needed, the equivalent grade, and whether the target is secured, reachable, tight, or out of reach.

Main features

FEATURE	WHAT IT DOES
Credit-weighted GPA	Cumulative and per-semester GPA on the 4.00 scale, never a plain average of grades.
Degree class	Classifies against First Class 3.70, Second Upper 3.30, Second Lower 3.00, General Pass 2.00, and states the gap to the next class up.
Target planner	Solves for the average required over the remaining credits, converts it to a grade, and reports secured, reachable, tight or out of reach rather than printing an impossible number.
Trend chart	Cumulative GPA after each semester as an SVG line chart with a dashed target line and a hover readout. Drawn from scratch, no charting library.
Repeat handling	A superseded attempt can be excluded from the GPA without being deleted from the record.
Non-GPA grades	MC, W and I carry credits but stay out of the GPA arithmetic.
Local storage	The transcript is saved in the browser and restored on the next visit. Export and import as JSON.
Example on first run	A new visitor lands on a sample transcript, clearly labelled, so the application is doing something the moment it opens.
Themes and access	Light and dark themes, keyboard operable, labelled form controls, live regions for status messages, responsive down to phone width.

Technologies and tools

Built with

- HTML, CSS and vanilla JavaScript. No framework, no build step, no dependencies at runtime.
- CSS custom properties for the light and dark token sets.

AI and other tools

- An AI coding agent in agent mode, running inside Visual Studio Code.
- jsdom, to drive the real DOM in the journey tests.

- Hand-written inline SVG for the chart.
- `localStorage` for persistence.
- Google Fonts: Newsreader, Public Sans, IBM Plex Mono, each with a local fallback stack.
- Node.js for the test runners and the single-file build script.
- Brave headless, to render and screenshot the page during review.
- A hosted artifact page, to give the application a public URL.

Prompts used

The application was produced from a single instruction issued to the coding agent, with the assignment brief attached as a file. There were no follow-up prompts: every subsequent step below was taken by the agent on its own.

PROMPT 1 - TYPED TO THE AI CODING AGENT, AGENT MODE

read @instructions.md and do it

ATTACHED INPUT - INSTRUCTIONS.MD

The assignment brief for Laboratory Exercise 1 - Part 1 (AI Web Application Challenge), supplied as a file in the working directory and read by the agent as its specification.

Skills the agent loaded on its own

The agent loaded three packaged instruction sets of its own accord during the run. These are AI-generated inputs in the same sense as a prompt, so they are recorded here.

- `dataviz` - chart form, colour and interaction rules, used for the GPA trend chart.
- `artifact-design` - design fundamentals, used for the typography and palette pass.
- `artifact-capabilities` - the runtime contract for the published page, used to make Export JSON work inside the hosted viewer.

Other AI-generated actions

The agent worked autonomously through the following sequence. Nothing in this list came from a further human instruction.

1. Read the brief, chose the problem, users and essential journey.
2. Wrote the GPA engine as pure functions with no DOM access, so it could be tested directly.
3. Wrote fourteen engine tests and ran them. Two failed. Both were investigated: one was a wrong expectation in the test, one was a real tie-break defect where a required average of 4.00 returned A+ instead of A. The defect was fixed with an explicit grade ladder; the wrong expectation was corrected.
4. Wrote the markup, the stylesheet and the interface layer.
5. Installed jsdom and wrote sixteen journey tests that drive the real DOM: submit the form, edit, exclude, delete, import, clear, read the planner verdict, count the chart points.

6. Rendered the page in headless Brave and inspected the screenshots. Three defects were found by looking rather than by testing: the chart wasted half its height on an unused range, the target label collided with the last data point at some targets, and the theme button reported the wrong mode under a dark operating system.
7. Found and fixed a theme bug where the application erased a theme already set by the host page.
8. Replaced the system font stack with a deliberate pairing, and shifted the neutral palette to sit with the accent colour.
9. Made the first run land on a labelled example transcript instead of an empty form.
10. Published the application as a hosted artifact, then discovered that the hosted viewer blocks ordinary download links, so Export JSON was routed through the host save API with the local download kept as the fallback. Republished.
11. Wrote the README, the single-file build script, this document and the PDF.

What was tested before submission

TEST	HOW	RESULT
GPA engine, 14 assertions	<code>node tests/run.js</code>	14 / 14 passed
User journey through a real DOM, 16 assertions	<code>node tests/ui.test.js</code> with jsdom	16 / 16 passed
Engine tests in a browser	<code>tests/tests.html</code>	Same 14 assertions pass
Visual check, light and dark	Headless Brave screenshots at 1280px	Checked, three layout defects fixed
Single-file build	Rendered <code>dist/gradepath.html</code> on its own	Identical to the multi-file version
Published page	Opened the artifact URL	Loads with the example transcript

Cases covered by those tests

- Credit weighting: four credits at A with one at D gives 3.40, not the 2.50 a plain average would give.
- An empty transcript reports no GPA rather than 0.00.
- An E counts in the GPA and earns no credit; MC and W stay out of the GPA but keep their credits.
- An excluded repeat attempt leaves every total.
- Class boundaries at the exact values: 3.70 is First Class, 3.69 is Second Upper.
- Semesters are listed in curriculum order however they were entered.
- The planner refuses zero remaining credits and names an unreachable target instead of printing an impossible average.
- The form rejects a missing grade, zero credits and an absurd credit value with a message that says how to fix it.
- The transcript survives a reload, and clearing it empties every total.

Known limits

- Data lives in one browser on one device. Clearing site data removes it. There is no account and no sync.
- The grade points and class boundaries are the BICT ones. Another programme edits the constants at the top of `assets/gpa.js`.
- The planner assumes the remaining credits all count towards the GPA.
- Figures should be checked against the official academic record before any decision.

Access and run instructions

Application URL	<code>https://claude.ai/code/artifact/d5a4d019-3de4-41af-bb9a-5b84002c5d93</code> Private by default. Open it and use the page's share menu to make it reachable, or run the files in the archive.
Run locally	Open <code>gradepath/index.html</code> in any modern browser. No server, no install, no configuration.
Single file	<code>gradepath/dist/gradepath.html</code> is the whole application in one file.
Run the tests	<code>node tests/run.js</code> , then <code>npm i jsdom</code> and <code>node tests/ui.test.js</code> .
Credentials	None. The application has no accounts, no keys and no secrets.
